Received: 30 October 2023 | Revised: 29 April 2024 | Accepted: 3 August 2024

DOI: 10.33012/navi.667

REGULAR PAPERS

Improving GNSS Positioning Correction Using Deep Reinforcement Learning with an Adaptive Reward Augmentation Method

Jianhao Tang[1,2] | **Zhenni Li***[1,2] | **Kexian Hou**[1] | **Peili Li**[1] | **Haoli Zhao**[2,4] | **Qianming Wang**[1,3] | **Ming Liu**[5] | **Shengli Xie**[6,7]

[1] School of Automation, Guangdong University of Technology, Guangzhou, China.

[2] Guangdong-HongKong-Macao Joint Laboratory for Smart Discrete Manufacturing, Guangzhou, China.

[3] Techtotop Microelectronics Technology Co., Ltd., Guangzhou, China.

[4] School of Artificial Intelligence, Hebei University of Technology, Tianjin, China.

[5] Robotics and Autonomous Systems (ROAS) Thrust, Hong Kong University of Science and Technology, Guangzhou, China.

[6] Key Laboratory of Intelligent Detection and the Internet of Things in Manufacturing, Guangzhou, China.

[7] Guangdong Key Laboratory of Internet of Things Information Technology, Guangzhou 510006, China.

Correspondence
Zhenni Li
School of Automation,
Guangdong University of Technology,
Guangzhou 510006, China.
Email: lizhenni2012@gmail.com

Abstract

High-precision global navigation satellite system (GNSS) positioning for automatic driving in urban environments remains an unsolved problem because of the impact of multipath interference and non-line-of-sight reception. Recently, methods based on data-driven deep reinforcement learning (DRL), which are adaptable to nonstationary urban environments, have been used to learn positioning-correction policies without strict assumptions about model parameters. However, the performance of DRL relies heavily on the amount of training data, and high-quality, available GNSS data collected in urban environments are insufficient because of issues such as signal attenuation and large stochastic noise, resulting in poor performance and low training efficiency for DRL. In this paper, we propose a DRL-based positioning correction method with an adaptive reward augmentation method (ARAM) to improve the GNSS positioning accuracy in nonstationary urban environments. To address the problem of insufficient training data in the target domain environment, we leverage sufficient data collected in source domain environments to compensate for insufficient training data, where the source domain environments can be in different locations than the target environment. We then employ ARAM to achieve domain adaptation that adaptively modifies data matching between the source domain and target domain by a simple modification to the reward function, thus improving the performance and training efficiency of DRL. Hence, our novel DRL model can achieve an adaptive dynamic-positioning correction policy for nonstationary urban environments. Moreover, the proposed positioning-correction algorithm can be flexibly combined with different model-based positioning approaches. The proposed method was evaluated using the Google smartphone decimeter challenge data set and the Guangzhou GNSS measurement data set, with results demonstrating that our method can obtain an improvement of approximately 10% in positioning performance over existing model-based methods and 8% over learning-based approaches.

Keywords

adaptive reward augmentation method, deep reinforcement learning, global navigation satellite system

Licensed under CC-BY 4.0 © 2024 Institute of Navigation

1 | INTRODUCTION

In recent years, with the development and integration of intelligent control theory and information technology, automatic driving has gradually become a focus of the automobile industry, with accurate, real-time global navigation satellite system (GNSS) positioning being a key technology (Bo et al., 2022). Although Global Positioning System (GPS)-equipped devices can provide centimeter-level position resolution in open areas, the resolution accuracy can be up to 3–5 m in complex urban areas that include high-rise buildings, overpasses, and "urban forests." Positioning errors of more than 10 m can occur in severe multipath-affected areas, caused by effects from multipath interference (MI) and non-line-of-sight (NLOS) reception with GNSS signals (Yuan et al., 2022). In addition, the positioning accuracy of various types of GPS devices can differ greatly. To achieve effective GNSS positioning in complex urban areas, auxiliary hardware equipment is sometimes used to enhance the accuracy of GNSS positioning. Examples of such equipment include inertial measurement units (IMUs) and reference stations. However, the quality of GNSS dominates such integrated GNSS/IMU navigation systems, whereas the deployment of reference stations is costly and time-consuming (Min et al., 2022; Zhang & Masoud, 2020). Moreover, although the 5G networks developed in recent years can improve the accuracy and availability of GNSS positioning technology, these networks require more costly base stations, and their multiple frequency bands may have negative effects on GNSS positioning (Liu & Guo, 2021).

To reduce the cost and difficulty of achieving high-precision GNSS positioning, various methods based on software algorithms have been proposed in recent years. For example, model-based methods such as those based on the Kalman filter (KF) and weighted least squares (WLS) have been shown to offer significant improvements in positioning accuracy. However, these methods rely on strict prior parameter assumptions and require manual tuning of covariances and other parameters (Wang et al., 2021). The recent development of learning-based methods is generating new ideas for improving positioning accuracy. These methods can model complex errors caused by MI/NLOS effects in urban environments using data and provide accurate positioning correction by using the powerful function-approximation capability of deep neural networks (DNNs), while requiring few strict assumptions about model parameters (Kanhere et al., 2021; H. Li et al., 2020; Z. Li et al., 2023). Additionally, recent studies (S. Li et al., 2023; Mohanty & Gao, 2023) have tightly combined learning-based methods with a model-based KF method to adaptively tune the parameters of noise covariance in model-based methods. However, the environment around a moving vehicle can change dramatically, particularly in nonstationary urban areas, and the distribution of input data for the learning model can exhibit substantial variations, making it challenging for DNN-based methods to adaptively correct the GNSS positioning in dynamically changing environments. In contrast to DNN-based methods that correct each position individually, deep reinforcement learning (DRL)-based methods can learn the dynamic pattern of positioning errors by interacting with the environment and considering the cumulative rewards of the positioning trajectory over time, enabling the model to evolve in response to changes in the environment and learn effective positioning-correction policies in a dynamically changing environment (Han et al., 2021; Zhang & Masoud, 2020; Zhao et al., 2023).

Although DRL-based positioning-correction methods have been shown to be effective in improving positioning accuracy, the performance of data-driven DRL models relies heavily on the amount of data available for training. Insufficient

data can lead to overfitting of the learning model to limited training data and failure to model the complex characteristics of the positioning errors, resulting in poor performance of the DRL model (Eysenbach et al., 2020; Liu et al., 2022). However, it is difficult to obtain high-quality data for a nonstationary urban environment because of issues such as response delay, signal interruption and attenuation, and large stochastic noise. Most GNSS positioning receivers can only receive measurements and solve positions with limited frequency (e.g., 1 Hz or 10 Hz); thus, a long duration is needed to collect enough data to train the model to achieve the expected performance, resulting in low training efficiency. Moreover, obtaining high-precision position data as reference labels for training requires costly and sophisticated measurement systems, such as high-precision map-matching systems.

In this paper, we propose a DRL-based GNSS positioning-correction method that incorporates the adaptive reward augmentation method (ARAM) to alleviate the problem of poor performance caused by insufficient data for training and to improve GNSS positioning accuracy in nonstationary urban environments. To address the problem of insufficient data for model training in the target domain environment, we leverage sufficient data collected in source domain environments to compensate for the limited training data corresponding to the target domain environment. The source domain environments can be in different locations than the target environment, aiming to improve the training efficiency and performance of the learning model. To achieve domain adaptation for the data between the target domain and source domain environments, we employ an ARAM (Eysenbach et al., 2020) that requires only a simple modification to the reward function (without complex, high-dimensional calculations) to adaptively modify data matching between the source domain and target domain. Hence, our novel DRL algorithm for GNSS positioning correction can improve GNSS positioning in urban environments with insufficient training data. Moreover, the proposed positioning-correction algorithm can be flexibly combined with different positioning approaches, such as the single point positioning KF, differential positioning real-time kinematic (RTK), and GNSS/IMU integration approaches, without complex model designing or high costs for hardware and computing resources. Experiments were conducted on the public Google smartphone decimeter challenge 2022 (GSDC 2022) data set (Fu et al., 2020) with pseudorange measurements and the collected Guangzhou GNSS measurement data set with carrier measurements, which demonstrate the effectiveness of the proposed method for different GNSS positioning systems. The main contributions of this work can be summarized as follows:

1. To address the poor performance and low training efficiency of the learning model caused by insufficient training data, we propose to leverage additional sufficient data collected in source domain environments to compensate for insufficient training data in the target domain environment.
2. To transfer the learned policy from the source domain to the target domain, we employ ARAM to achieve domain adaptation by a simple modification to the reward function, thereby adaptively modifying the matching of data between the source domain and target domain.
3. Based on ARAM and the DRL model, we develop a novel DRL-based GNSS positioning-correction algorithm to improve positioning accuracy in environments with insufficient training data. Moreover, the proposed algorithm can be flexibly combined with different positioning methods such as the single point positioning KF and differential positioning RTK methods.

The remainder of this paper is organized as follows. In Section 2, we present the preliminaries related to DRL and GNSS positioning. In Section 3, we provide a description of the proposed algorithm for GNSS positioning correction. In Section 4, we give details of our experimental results for positioning correction using real-world data sets. Finally, we present our conclusions in Section 5.

2 | PRELIMINARIES

2.1 | Background

(1) DRL problem: Reinforcement learning (RL) can automatically acquire optimal behavioral policies for complex decision-making tasks through interactions with the environment. In our study, we take the GNSS positioning-correction problem as an RL environment to be solved. Because we cannot observe complete information of the vehicle driving environment, the positioning-correction problem can be modeled as a partially observable Markov decision process (POMDP), in which the observation only contains partial environment information. The framework of the environment can be specified by a tuple $M := (O, A, T, R, \gamma)$, where O is the observation space that specifies the current state of the vehicle agent, A is the action of the positioning-correction operation, T is the transition probability, R is the reward function that evaluates the effectiveness of the action, and $\gamma \in [0,1]$ is a discount factor. An RL trajectory with T steps denoted as $\tau = \{(\mathbf{o}_t, \mathbf{a}_t r_t, \mathbf{o}_{t+1})\}_{t=1}^{T}$ indicates that an agent with an observation $\mathbf{o}_t \in O$ at step t takes an action $\mathbf{a}_t \in A$ and then obtains a reward $\mathbf{r}_t \in R$ from the environment and the observation $\mathbf{o}_t$ transitions to $\mathbf{o}_{t+1} \in O$ at the next step $t+1$ according to the transition probability $T(\mathbf{o}_{t+1} \mid \mathbf{o}_t, \mathbf{a}_t)$. Given a discount factor γ, our goal is to learn an optimal policy π that maximizes the discounted accumulated reward R_τ along a trajectory τ, i.e., $R_\tau = \sum_{t=0}^{T} \gamma^t r_{t+1}$, which indicates that the positioning error along a vehicle trajectory is expected to be minimized.

To learn the optimal policy, we also define the observation value as the expected accumulated reward obtained by the policy π in $\mathbf{o}_t$, i.e., $V^\pi(\mathbf{o}_t) := \mathbb{E}\left(\sum_{t=0}^{T} \gamma^t r_{t+1} \mid \mathbf{o} = \mathbf{o}_t\right)$, where the agent can explore the optimal policy based on $V^\pi(\mathbf{o}_t)$. In practice, state and action spaces may be prohibitively large and continuous. To deal with this problem, we typically use DNNs to predict actions and observation values; thus, a typical RL problem can be approached as a DRL problem.

(2) Domain adaptation: Although DRL shows promising performance in complex decision-making tasks, training of the data-driven DRL model requires a large amount of data, and obtaining high-quality data from some specific target domain environments can be difficult and costly. To address this issue, we can access a source domain environment with a structure similar to that of the target domain, where the data from the source domain are more abundant and easier to obtain than target domain data, such as a simulator and an offline data set with sufficient data. However, transferring a learned policy from the source domain to the target domain is challenging because the optimal policy learned in the source domain may be suboptimal in the target domain. Therefore, domain adaptation is required for transferring sufficient experiences from the source domain to the training of the target domain.

The domain adaptation considers two domain environments: the target domain environment $\mathcal{E}_{\text{target}}$ (e.g., a real-world environment in which data are difficult to obtain) and the source domain environment $\mathcal{E}_{\text{source}}$ (e.g., a simulator and an offline

data set with sufficient data). These two domain environments should have the same observation space O, action space A, reward function R, and initial observation probability $p(\mathbf{o}_1)$. The objective of domain adaptation is to learn a policy π to achieve high rewards in the target domain environment $\mathcal{E}_{\text{target}}$ by interacting with the source domain environment $\mathcal{E}_{\text{source}}$ while including as few interactions with the target domain environment $\mathcal{E}_{\text{target}}$ as possible.

2.2 | Related Work

In a typical GNSS positioning flow, the GNSS device first receives GNSS measurements from multiple satellites, including the transmitting satellite position and the signal transmission time. Then, a rough position of the GNSS device can be obtained by applying conventional positioning methods to the received GNSS measurements, such as model-based KF, WLS, and RTK methods. To achieve accurate positioning of GNSS devices, high-quality signals must be received from at least four satellites. In most cases, GNSS devices can receive the minimum required number of satellite signals; however, the signal quality will be significantly degraded in urban areas with high-rise buildings or dense vegetation because of MI/NLOS effects, resulting in large biases in GNSS positioning. MI/NLOS effects are time-varying, nonlinear, and non-Gaussian, and the influencing factors are complex, making it difficult to model MI/NLOS error distributions with traditional methods based on linear or Gaussian assumptions (Zhang & Masoud, 2020). For some hardware-based positioning approaches, a common choice is to use auxiliary hardware equipment to improve the positioning performance of GNSS devices, for example, by integrating GNSS and an inertial navigation system (INS), which can provide the velocity, attitude, and high-rate position of vehicles (Niu et al., 2022; Zhu et al., 2022). Although such GNSS/INS integration can improve positioning accuracy to a certain extent, the GNSS quality still dominates the positioning performance of such integrated navigation systems (Sun et al., 2023). Meanwhile, the failure of auxiliary hardware equipment will degrade the positioning performance of the system (Zhang & Masoud, 2020).

In recent years, many software-based studies have attempted to improve the accuracy of GNSS positioning in complex urban environments. The learning-based deep learning method has achieved remarkable success in correcting complex positioning errors (Zhao, Wang, et al., 2022; Zhao, Wu, et al., 2022). Kanhere et al. (2021) developed a set-based transformer model to model complex MI/NLOS errors with the GNSS measurements of multiple satellites, using the line-of-sight (LOS) vector and residuals as input features to obtain accurate position corrections. Mohanty and Gao (2022) further attempted to learn the connection aspects of different satellite measurement features via a graph convolutional neural network (GCNN) and proposed a hybrid framework combining a learning-based GCNN method and a model-based KF method to learn accurate position corrections via GNSS measurements from smartphones. However, the input feature distributions of dynamically changing driving scenarios exhibit substantial variations across different environments and motion states, making it challenging for DNN-based methods to learn adaptive positioning-correction policies in response to rapid environmental changes (Zhang et al., 2019; Zhao et al., 2023). In addition, the DRL-based method can learn optimal policies for positioning correction by interacting with the environment, enabling it to adapt to rapidly changing non-stationary environments (Han et al., 2021; Zhang et al., 2019). Zhao et al. (2023) proposed using a long short-term memory (LSTM) module to extract temporal

features from the time-series trajectory and to thus improve the GNSS positioning accuracy. However, because of issues such as large stochastic noise, signal blocking, and attenuation, it is difficult to provide sufficient high-quality training data for data-driven DRL models, which can cause poor performance and low training efficiency for DRL (Eysenbach et al., 2020; Liu et al., 2022). Our study is the first to address the problem of insufficient training data for learning-based GNSS positioning methods.

3 | DRL FOR GNSS POSITIONING CORRECTION WITH ARAM

In this section, we first set up an RL environment for GNSS positioning correction. Then, we construct a DRL model to learn an optimal policy for positioning correction. In the training phase, model training is assumed to be conducted in a specific environment with data obtained by different receivers, and the initial positions in the target environment can then be corrected after training. In the testing phase, the learned model is further evaluated in other target environments, and the initial positions can be corrected in real time.

3.1 | RL Environment for GNSS Positioning Correction

In this subsection, we present the details of the RL environment for positioning correction, including the definition of the observation space, action space, reward function, and LSTM feature extractor.

(1) Observation space setting: Existing methods use the vehicle position as the observation, which ignores the complex environmental errors caused by MI/NLOS effects in urban environments. To accurately specify the current state of the vehicle agent, we introduce GNSS measurements for the observation. We first denote the number of visible satellites at time t as S_t, and the associated GNSS measurement set at time t consisting of pairs of satellite positions and corresponding pseudoranges can be defined as follows:

$$\mathcal{M}_t = \{(\mathbf{p}_t^{(1)}, \rho_t^{(1)}), (\mathbf{p}_t^{(2)}, \rho_t^{(2)}), ..., (\mathbf{p}_t^{(S_t)}, \rho_t^{(S_t)})\}, \quad \forall\, t \in \{1, ..., T\} \tag{1}$$

where $\{\mathbf{p}_t^{(1)}, ..., \mathbf{p}_t^{(S_t)}\}$ are the estimated visible satellite positions at time t and $\{\rho_t^{(1)}, ..., \rho_t^{(S_t)}\}$ are the corresponding pseudoranges. Then, the initial rough position of the GNSS device $\mathbf{POS}_t^{\text{init}}$ can be obtained by conventional positioning methods Ψ, such as KF and WLS methods, with the measurement set $\mathcal{M}_t$, which can be defined as follows:

$$\mathbf{POS}_t^{\text{init}} = \Psi(\mathcal{M}_t) = (\mathrm{X}_t, \mathrm{Y}_t, \mathrm{Z}_t), \quad \forall\, t \in \{1, ..., T\} \tag{2}$$

where X_t, Y_t, and Z_t are the values of different directions in the earth-centered, earth-fixed (ECEF) coordinate system of the initial position. However, because of MI/NLOS interference, the initial position usually contains biases in meters from the ground true position.

To consider the influence of MI/NLOS on positioning, we introduce GNSS features as observations of the positioning-correction environment to model complex environmental errors, including the normalized LOS vector **LOS** denoting the

relative position of each satellite and the pseudorange residual **RES** denoting the difference between the expected pseudorange and the measured pseudorange:

$$\mathbf{LOS}_t^{(i)} = \frac{\mathbf{p}_t^{(i)} - \mathbf{POS}_t^{\text{init}}}{\| \mathbf{p}_t^{(i)} - \mathbf{POS}_t^{\text{init}} \|_2}, \qquad \mathbf{RES}_t^{(i)} = \rho_t^{(i)} - \| \mathbf{p}_t^{(i)} - \mathbf{POS}_t^{\text{init}} \|_2, \quad \forall t \in \{1, ..., T\},\ i \in \{1, ..., S_t\} \tag{3}$$

By concatenating the LOS vectors and the pseudorange residual, the GNSS features are formed as the observation of the environment:

$$\mathbf{o}_t := \left[\mathcal{L}_t^T \quad , \mathcal{R}_t^T \right] \tag{4}$$

where $\mathcal{L}_t = \left[\mathbf{LOS}_t^{(1)}, ..., \mathbf{LOS}_t^{(S_{max})} \right]$, $\mathcal{R}_t = \left[\mathbf{RES}_t^{(1)}, ..., \mathbf{RES}_t^{(S_{max})} \right]$, and S_{max} represents the maximum number of visible satellites for the vehicle trajectories. The number of visible satellites at each time t for the vehicle trajectory may differ, and thus, the elements of $\mathcal{L}_t$ and $\mathcal{R}_t$ corresponding to non-visible satellites are filled with zeros. The LOS vector provides orientation information from the receiver to different satellites, which is correlated with the positioning errors arising from MI/NLOS effects, indicating the spatial characteristics of satellites at time t. The pseudorange residual provides insights into potential errors during the estimation of the model-based positioning method and GNSS measurement.

(2) Action space setting: We define the action as a correction operation to the initial position of the GNSS device $\mathbf{POS}_t^{\text{init}} = (\mathrm{X}_t, \mathrm{Y}_t, \mathrm{Z}_t)$. In addition, if there is a large difference between the scale of the geodetic surface error and the elevation error, the longitude and latitude of the position can be considered as the only corrected elements, that is, the reference system can be converted from ECEF to geodetic coordinates, and the initial position for correction can be defined as $\mathbf{POSLL}_t^{\text{init}} = (\mathrm{Lat}_t, \mathrm{Lon}_t)$. The action space is set to continuous to achieve a higher-precision position correction and to avoid the learning difficulty caused by a large number of discrete actions. This procedure is executed as follows:

First, the output of the actor is defined as the mean and standard deviation of the Gaussian distribution for the values of the positioning correction for each direction of the ECEF coordinate system, i.e., $\mathbf{a}_t := \mathbf{a}_t^{\text{ECEF}} = \left(\mu_{\mathrm{X}_t}, \sigma_{\mathrm{X}_t}, \mu_{\mathrm{Y}_t}, \sigma_{\mathrm{Y}_t}, \mu_{\mathrm{Z}_t}, \sigma_{\mathrm{Z}_t} \right)$. Similarly, if only the longitude and latitude of the positions are considered to be corrected, the action can be defined as the mean and standard deviation of the Gaussian distribution for the positioning correction of the latitude and longitude, i.e., $\mathbf{a}_t := \mathbf{a}_t^{\text{LL}} = \left(\mu_{\mathrm{Lat}_t}, \sigma_{\mathrm{Lat}_t}, \mu_{\mathrm{Lon}_t}, \sigma_{\mathrm{Lon}_t} \right)$.

Second, the continuous action for the positioning-correction operation is sampled from the Gaussian distribution based on the output of the actor and is clipped to a maximum absolute value m, i.e., $\Delta X \sim \mathcal{N}\left(\mu_{\mathrm{X}_t}, \sigma_{\mathrm{X}_t}^2 \right)$, $\Delta Y_t \sim \mathcal{N}\left(\mu_{\mathrm{Y}_t}, \sigma_{\mathrm{Y}_t}^2 \right)$, $\Delta Z_t \sim \mathcal{N}\left(\mu_{\mathrm{Z}_t}, \sigma_{\mathrm{Z}_t}^2 \right)$, or $\Delta Lat_t \sim \mathcal{N}\left(\mu_{\mathrm{Lat}_t}, \sigma_{\mathrm{Lat}_t}^2 \right)$, $\Delta Lon_t \sim \mathcal{N}\left(\mu_{\mathrm{Lon}_t}, \sigma_{\mathrm{Lon}_t}^2 \right)$, and $|\Delta X_t|, |\Delta Y_t|, |\Delta Z_t| < m$ or $|\Delta Lat_t|, |\Delta Lon_t| < m$.

Finally, with a scaling factor defined for the correction operation u^{ECEF} or u^{LL}, the output of the corrected position can be calculated as $\hat{\mathbf{POS}}_t = (\mathrm{X}_t + u^{\text{ECEF}} \Delta X_t, \mathrm{Y}_t + u^{\text{ECEF}} \Delta Y_t, \mathrm{Z}_t + u^{\text{ECEF}} \Delta Z_t)$ or $\hat{\mathbf{POSLL}}_t = (\mathrm{Lat}_t + u^{\text{LL}} \Delta Lat_t, \mathrm{Lon}_t + u^{\text{LL}} \Delta Lon_t)$.

(3) Reward function setting: To accurately evaluate the effectiveness of the action for positioning correction, we employ the correction advantage error as the reward function of the environment, defined as follows:

$$r_t := r_t^{\text{ECEF}} = \| \mathbf{POS}_t^{\text{init}} - \mathbf{POS}_t^{\text{ref}} \|_2 - \| \hat{\mathbf{POS}}_t - \mathbf{POS}_t^{\text{ref}} \|_2 \tag{5}$$

$$r_t := r_t^{\text{LL}} = \text{Vincenty}(\mathbf{POSLL}_t^{\text{init}}, \mathbf{POSLL}_t^{\text{ref}}) - \text{Vincenty}(\hat{\mathbf{POSLL}}_t, \mathbf{POSLL}_t^{\text{ref}}) \tag{6}$$

where r_t^{ECEF} and r_t^{LL} are the reward functions for the ECEF coordinate system and the latitude and longitude, respectively, $\mathrm{Vincenty}(\cdot)$ is Vincenty's formula (Vincenty, 1975) for calculating the distance from the latitude and longitude, and $\mathbf{POS}_t^{\mathrm{ref}}$ and $\mathbf{POSLL}_t^{\mathrm{ref}}$ are the reference positions obtained by high-precision but costly positioning methods such as map-matching methods and reference stations. Hence, we can select $r_t := r_t^{\mathrm{ECEF}}$ or $r_t := r_t^{\mathrm{LL}}$ when we consider the positioning correction of the ECEF coordinate system or the correction of the latitude and longitude of the positions. In contrast to simply using the mean squared error (Zhang & Masoud, 2020) to calculate the accuracy of positioning as the reward function, the correction advantage error used herein can better reflect the advantages of a good correction policy compared with the initial positioning method.

(4) LSTM feature extractor: To leverage the historical information of the observations and to extract temporal aspects from observations of the POMDP model, which cannot reflect the complete states of the environment, we employ the LSTM feature extractor $f_{\theta_{\mathrm{lstm}}}$ with a parameter set θ_{lstm} to process the input observation, where the forward propagation of the LSTM feature extractor consists of the block input, input gate, forget gate, cell gate, and hidden output gate. Given an input observation $\mathbf{o}_t \in \mathbb{R}^{S_{max}\times 4}$, the hidden state of the LSTM feature extractor output can be defined as $\mathbf{h}_t = f_{\theta_{\mathrm{lstm}}}(\mathbf{o}_t)$.

Through the processing of the LSTM feature extractor, we can take the output hidden state $\mathbf{h}_t$ as the fully observable belief state to replace the original partial observation $\mathbf{o}_t$. Therefore, the original POMDP problem with the partial observation trajectory $\tau = \{(\mathbf{o}_t, \mathbf{a}_t, r_t, \mathbf{o}_{t+1})\}_{t=1}^{T}$ can be converted to a Markov decision process problem with the belief state trajectory $\hat{\tau} = \{(\mathbf{h}_t, \mathbf{a}_t, r_t, \mathbf{h}_{t+1})\}_{t=1}^{T}$ to learn an optimal policy with RL algorithms.

3.2 | LSTM Proximal Policy Optimization Combined with ARAM for GNSS Positioning Correction

In this subsection, we construct a DRL model for GNSS positioning correction. We first apply ARAM to address the problem of limited data in the target domain environment and then adopt the proximal policy optimization (PPO) model to optimize the advantage clipping policy in the trust region. Overall, we develop an LSTMPPO algorithm based on ARAM (LSTMPPO-ARAM) or GNSS positioning correction.

(1) ARAM for improving DRL performance: In nonstationary urban environments, it is difficult to collect data for GNSS positioning because of issues such as response delay, signal interruption, and attenuation. Insufficient data can lead to overfitting of the model to limited training data and poor performance. To address this problem, we propose to transfer experiences from a source domain environment with sufficient offline data, aiming to compensate for the poor performance caused by insufficient data from the target domain. Moreover, to transfer the learned policy from the source domain to the target domain, we propose to employ ARAM to achieve dynamic domain adaptation by a simple adaptive reward augmentation.

Considering the reward as a desired distribution over positioning trajectories according to the probabilistic inference for RL, our goal is to learn an optimal policy for each observation to maximize the accumulated reward in the source domain environment $\mathcal{E}_{\mathrm{source}}$ and target domain environment $\mathcal{E}_{\mathrm{target}}$. We first define the desired distribution for the trajectories with the optimal policy π as follows:

$$\mathbb{E}_t\left[\exp\left(\sum_t r_t(\mathbf{o}_t, \mathbf{a}_t)/\eta\right)\right] = p(\mathbf{o}_1)\left(\prod_t T(\mathbf{o}_{t+1} \mid \mathbf{o}_t, \mathbf{a}_t)\right)\exp\left(\sum_t r_t(\mathbf{o}_t, \mathbf{a}_t)/\eta\right) \quad (7)$$

where $\mathbb{E}_t[\cdot]$ denotes the average over a finite sampling batch, $p(\mathbf{o}_1)=1$ is the initial observation probability of both domains, where the initial positions and observations are assumed to be identical, and η is a temperature parameter. The distribution for positioning trajectories $q^{\pi}(\tau)$ can be defined as follows:

$$q^{\pi}(\tau)=p(\mathbf{o}_1)\prod_t T(\mathbf{o}_{t+1}\,|\,\mathbf{o}_t,\mathbf{a}_t)\,\pi(\mathbf{a}_t\,|\,\mathbf{o}_t) \tag{8}$$

Next, we derive the risk-sensitive reward objective (Mihatsch & Neuneier, 2002) by associating the transition probabilities of the two domains and further derive a lower bound on this objective by using Jensen's inequality (Eysenbach et al., 2020) as follows:

$$\begin{aligned}
&\log\mathbb{E}_{\tau\sim q^{\pi}_{\text{target}}(\tau)}\left[\exp\left(\sum_t r_t(\mathbf{o}_t,\mathbf{a}_t)/\eta\right)\right]=\log\mathbb{E}_{\tau\sim q^{\pi}_{\text{source}}(\tau)}\left[\frac{q^{\pi}_{\text{target}}(\tau)}{q^{\pi}_{\text{source}}(\tau)}\exp\left(\sum_t r_t(\mathbf{o}_t,\mathbf{a}_t)/\eta\right)\right]\\
&=\log\mathbb{E}_{\tau\sim q^{\pi}_{\text{source}}(\tau)}\left[\left(\prod_t\frac{T_{\text{target}}(\mathbf{o}_{t+1}\,|\,\mathbf{o}_t,\mathbf{a}_t)}{T_{\text{source}}(\mathbf{o}_{t+1}\,|\,\mathbf{o}_t,\mathbf{a}_t)}\right)\exp\left(\sum_t r_t(\mathbf{o}_t,\mathbf{a}_t)/\eta\right)\right]\\
&=\log\mathbb{E}_{\tau\sim q^{\pi}_{\text{source}}(\tau)}\left[\exp\left(\sum_t r_t(\mathbf{o}_t,\mathbf{a}_t)/\eta+\Delta r_t(\mathbf{o}_{t+1},\mathbf{o}_t,\mathbf{a}_t)\right)\right]\\
&\geq\mathbb{E}_{\tau\sim q^{\pi}_{\text{source}}(\tau)}\left[\sum_t\left(r_t(\mathbf{o}_t,\mathbf{a}_t)/\eta+\Delta r_t(\mathbf{o}_{t+1},\mathbf{o}_t,\mathbf{a}_t)\right)\right]
\end{aligned} \tag{9}$$

where $\Delta r_t(\mathbf{o}_{t+1},\mathbf{o}_t,\mathbf{a}_t)\triangleq\log T_{\text{target}}(\mathbf{o}_{t+1}\,|\,\mathbf{o}_t,\mathbf{a}_t)-\log T_{\text{source}}(\mathbf{o}_{t+1}\,|\,\mathbf{o}_t,\mathbf{a}_t)$ is the reward augmentation for the source domain obtained by transition probabilities. Thus, taking time $t\to\infty$ for each rollout, the original RL problem can be converted to an inference problem: $\arg\max_{\pi}\log\mathbb{E}_{\tau\sim q^{\pi}_{\text{target}}(\tau)}\left[\exp\left(\sum_t r_t(\mathbf{o}_t,\mathbf{a}_t)/\eta\right)\right]$, which can be further stated as a maximum of the lower bound by introducing the discount factor γ:

$$\mathbb{E}_{\tau\sim q^{\pi}_{\text{source}}(\tau)}\left[\sum_t^{\infty}\gamma^t\left(r_t(\mathbf{o}_t,\mathbf{a}_t)+\eta\Delta r_t(\mathbf{o}_{t+1},\mathbf{o}_t,\mathbf{a}_t)\right)\right] \tag{10}$$

The objective in Equation (10) suggests that we can transfer experiences from the source domain to the target domain, achieving domain adaptation by modifying the reward function of the source domain with the reward augmentation Δr_t, i.e., $r_t\leftarrow r_t+\eta\Delta r_t$. Intuitively, the reward augmentation Δr_t accounts for the discrepancy between the source domain and the target domain.

In practice, the reward augmentation Δr_t can be obtained by adopting two binary classifiers $\mathcal{C}_{\text{oao}}(\cdot\,|\,\mathbf{o}_t,\mathbf{a}_t,\mathbf{o}_{t+1})$ and $\mathcal{C}_{\text{oa}}(\cdot\,|\,\mathbf{o}_t,\mathbf{a}_t)$ used to predict whether experiences come from the source domain or the target domain. Hence, the reward augmentation can be adaptively estimated with the trained $\mathcal{C}_{\text{oao}}(\cdot\,|\,\mathbf{o}_t,\mathbf{a}_t,\mathbf{o}_{t+1})$ and $\mathcal{C}_{\text{oa}}(\cdot\,|\,\mathbf{o}_t,\mathbf{a}_t)$ by using data, to achieve an adaptive reward augmentation without prior assumptions about the source and target domain environments. This transformation relates the transition probabilities and classifier probabilities by Bayes' rule:

$$\Delta r_t=\log\frac{T_{\text{target}}(\mathbf{o}_{t+1}\,|\,\mathbf{o}_t,\mathbf{a}_t)}{T_{\text{source}}(\mathbf{o}_{t+1}\,|\,\mathbf{o}_t,\mathbf{a}_t)}=\log\frac{\mathcal{C}_{\text{oao}}(\text{target}\,|\,\mathbf{o}_t,\mathbf{a}_t,\mathbf{o}_{t+1})}{\mathcal{C}_{oao}(\text{source}\,|\,\mathbf{o}_t,\mathbf{a}_t,\mathbf{o}_{t+1})}-\log\frac{\mathcal{C}_{\text{oa}}(\text{target}\,|\,\mathbf{o}_t,\mathbf{a}_t)}{\mathcal{C}_{oa}(\text{source}\,|\,\mathbf{o}_t,\mathbf{a}_t)} \tag{11}$$

Therefore, we can transfer the experience learning in the source domain with sufficient data to compensate for the poor performance caused by insufficient data from the target domain and achieve domain adaptation by the adaptive reward augmentation Δr_t.

(2) Optimization of LSTMPPO-ARAM for positioning correction: The model consists of an actor network and a critic network; the actor network is used to output Gaussian distributions for the continuous actions, and the positioning correction is obtained by distribution sampling. The critic network is used to estimate the value of observations for learning the optimal policy.

We first construct the actor network π_{θ_a} and critic network V_{θ_c} with a multi-layer fully connected network architecture. The corresponding parameter sets of the actor and critic are θ_a and θ_c, which contain network weights and biases. Therefore, the action from the output of the actor network at time t can be denoted as follows:

$$\mathbf{a}_t = \pi_{\theta_a}(\mathbf{h}_t) = \pi_{\theta_a}(f_{\theta_{\text{lstm}}}(\mathbf{o}_t)) \tag{12}$$

The estimated value for the observation from the output of the critic network at time t can be defined as follows:

$$V^{\pi}(\mathbf{o}_t) \approx V_{\theta_c}(\mathbf{h}_t) = V_{\theta_c}(f_{\theta_{\text{lstm}}}(\mathbf{o}_t)) \tag{13}$$

Next, we apply the benchmark PPO algorithm for actor–critic learning to learn the optimal policy. To effectively quantify the quality of the output action $\mathbf{a}_t$ in the observation $\mathbf{o}_t$ and reduce the variance of the policy gradient, a generalized advantage estimation (Schulman et al., 2015) is used to calculate the policy advantage $\hat{A}_t(\mathbf{o}_t, \mathbf{a}_t) = \sum_{l=t}^{T-1} \gamma^{l-t}\delta_l$, where $\delta_l = r_l + \gamma V^{\pi}(\mathbf{o}_{t+1}) - V^{\pi}(\mathbf{o}_t)$. The rewards obtained in the source domain environment are modified with the adaptive reward augmentation Δr_t for domain adaptation. Then, the advantage $\hat{A}_t$ is introduced to the clipping objective for policy learning, and the loss function of the actor network can be defined as follows:

$$\mathcal{L}_a^{clip}(\boldsymbol{\theta}_a) = \mathbb{E}_t\left[\min\left(\text{ratio}_t(\boldsymbol{\theta}_a)\hat{A}_t, \text{clip}\left(\text{ratio}_t(\boldsymbol{\theta}_a), 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)\right] \tag{14}$$

where $\text{ratio}_t(\boldsymbol{\theta}_a) = \pi_{\boldsymbol{\theta}_a}(a_t \mid \mathbf{o}_t) / \pi_{\boldsymbol{\theta}_a^{\text{old}}}(a_t \mid \mathbf{o}_t)$ denotes the probability ratio of the new and old policies and ϵ is a small value for clipping. To approximate the accurate observation value for bootstrapping off the policy learning, the mean-squared return error (Le et al., 2017) is used to construct the loss function of the critic network by considering the discount accumulated reward along the trajectory with T steps, defined as follows:

$$\mathcal{L}_c(\boldsymbol{\theta}_c) = \mathbb{E}_t\left[\frac{1}{2}\left\|\sum_{l=t}^{l+T}\gamma^{l-t}r_{l+1} - V^{\pi}(\mathbf{o}_t)\right\|_2^2\right] \tag{15}$$

In the end, the model with the parameter set $\boldsymbol{\theta} = \{\boldsymbol{\theta}_{\text{lstm}}, \boldsymbol{\theta}_a, \boldsymbol{\theta}_c\}$ can be updated by minimizing the loss functions $\mathcal{L}_c(\boldsymbol{\theta}_c)$ and $\mathcal{L}_a^{clip}(\boldsymbol{\theta}_a)$ based on the Adam method (Kingma & Ba, 2015) using the experiences collected by interacting with the source and target domain environments. Additionally, the two classifiers $\mathcal{C}_{\text{oao}}$ and $\mathcal{C}_{\text{oa}}$ can be updated by minimizing the standard entropy loss:

$$\begin{aligned}\mathcal{L}_{\text{oao}}\left(\theta_{\text{oao}}\right) &= -\mathbb{E}_{\mathcal{D}_{\text{target}}}\left[\log \mathcal{C}_{\theta_{\text{oao}}}(\text{target}\,|\,\mathbf{o}_t, \mathbf{a}_t, \mathbf{o}_{t+1})\right] \\ &\quad -\mathbb{E}_{\mathcal{D}_{\text{source}}}\left[\log \mathcal{C}_{\theta_{\text{oao}}}(\text{source}\,|\,\mathbf{o}_t, \mathbf{a}_t, \mathbf{o}_{t+1})\right] \\ \mathcal{L}_{\text{oa}}\left(\theta_{\text{oa}}\right) &= -\mathbb{E}_{\mathcal{D}_{\text{target}}}\left[\log \mathcal{C}_{\theta_{\text{oa}}}(\text{target}\,|\,\mathbf{o}_t, \mathbf{a}_t)\right] \\ &\quad -\mathbb{E}_{\mathcal{D}_{\text{source}}}\left[\log \mathcal{C}_{\theta_{\text{oa}}}(\text{source}\,|\,\mathbf{o}_t, \mathbf{a}_t)\right]\end{aligned} \tag{16}$$

where $\boldsymbol{\theta}_{\text{oao}}$ and $\boldsymbol{\theta}_{\text{oa}}$ are the parameter sets of two classifiers $\mathcal{C}_{\text{oao}}$ and $\mathcal{C}_{\text{oa}}$ and define $\boldsymbol{\theta}_{\mathcal{C}} = \{\boldsymbol{\theta}_{\text{oao}}, \boldsymbol{\theta}_{\text{oa}}\}$.

In the training phase, the target domain environment is a specific environment, and the source domain environment can be in different locations. The objective is to improve the GNSS positioning in a target environment after several training iterations. For each training iteration, we first collect experiences of trajectories from the source domain environment and from the target domain environment after a period of iterations denoted as ratio r and store them in respective replay buffers, $\mathcal{D}_{\text{source}}$ and $\mathcal{D}_{\text{target}}$. Then, we sample a batch of data from both buffers to update the two classifiers, $\mathcal{C}_{\text{oao}}$ and $\mathcal{C}_{\text{oa}}$, with the Adam method by minimizing the standard entropy loss, $\mathcal{L}_{\text{oao}}\left(\theta_{\text{oao}}\right)$ and $\mathcal{L}_{\text{oa}}\left(\theta_{\text{oa}}\right)$. Finally, the LSTMPPO-ARAM model can be trained with the experiences from the two replay buffers, $\mathcal{D}_{\text{source}}$ and $\mathcal{D}_{\text{target}}$, where the rewards of the source replay buffer are modified by the adaptive reward augmentation Δr_t computed using the two classifiers, $\mathcal{C}_{\text{oao}}$ and $\mathcal{C}_{\text{oa}}$, with Equation (11). The LSTMPPO-ARAM algorithm is summarized in Algorithm 1.

In the testing phase, we test the learned policy for GNSS positioning in a target environment similar to the training environment, except that the trained LSTM feature extractor and actor network are required for real-time positioning correction. We input the observation at time t and obtain a positioning correction of

ALGORITHM 1
LSTMPPO-ARAM method for GNSS positioning correction

Input: Replay buffer capacity N, reward discount factor γ, temperature parameter η, scaling factor u^{ECEF} or u^{LL}, learning rate lr for RL training, learning rate α for classifier training, ratio r, and iteration number num.

Initialization: LSTMPPO-ARAM parameter set $\boldsymbol{\theta} = \{\boldsymbol{\theta}_{\text{lstm}}, \boldsymbol{\theta}_a, \boldsymbol{\theta}_c\}$, classifier parameter set $\boldsymbol{\theta}_{\mathcal{C}} = \{\boldsymbol{\theta}_{\text{oao}}, \boldsymbol{\theta}_{\text{oa}}\}$, replay buffers $\mathcal{D}_{\text{source}}$ and $\mathcal{D}_{\text{target}}$, source and target domain environments $\mathcal{E}_{\text{source}}$ and $\mathcal{E}_{\text{target}}$, and iteration step $iter = 1$.

Main iterations:

1: **for** $iter = 1, \ldots, num$ **do**

2: Collect experiences $\{(\mathbf{o}_t, \mathbf{a}_t, r_t, \mathbf{o}_{t+1})\}_{t=1}^{N}$ with the policy $\pi_{\boldsymbol{\theta}_a}$ from the source domain environment $\mathcal{E}_{\text{source}}$ and store in $\mathcal{D}_{\text{source}}$;

3: **if** $t \mod r == 0$ **do**

4: Collect experiences $\{(\mathbf{o}_t, \mathbf{a}_t, r_t, \mathbf{o}_{t+1})\}_{t=1}^{N}$ with the policy $\pi_{\boldsymbol{\theta}_a}$ from the target domain environment $\mathcal{E}_{\text{target}}$ and store in $\mathcal{D}_{\text{target}}$;

5: Update the classifier parameter set $\boldsymbol{\theta}_{\mathcal{C}}$ by minimizing the standard entropy loss $\mathcal{L}_{\text{oao}}(\boldsymbol{\theta}_{\text{oao}})$ and $\mathcal{L}_{\text{oa}}(\boldsymbol{\theta}_{\text{oa}})$ with learning rate α based on the adaptive moment estimation (Adam) method;

6: Calculate Δr_t from Equation (11); then, modify the reward collected in $\mathcal{D}_{\text{source}}$:

$$r_t \leftarrow r_t + \eta \Delta r_t$$

7: Update the LSTMPPO-ARAM parameter set $\boldsymbol{\theta}$ with experiences in $\mathcal{D}_{\text{target}}$ and $\mathcal{D}_{\text{source}}$ by minimizing the loss function $\mathcal{L}_a^{clip}(\boldsymbol{\theta}_a)$ and $\mathcal{L}_c(\boldsymbol{\theta}_c)$ with learning rate lr based on the Adam method.

Output: LSTMPPO-ARAM model with the parameter set $\boldsymbol{\theta} = \{\boldsymbol{\theta}_{\text{lstm}}, \boldsymbol{\theta}_a, \boldsymbol{\theta}_c\}$.

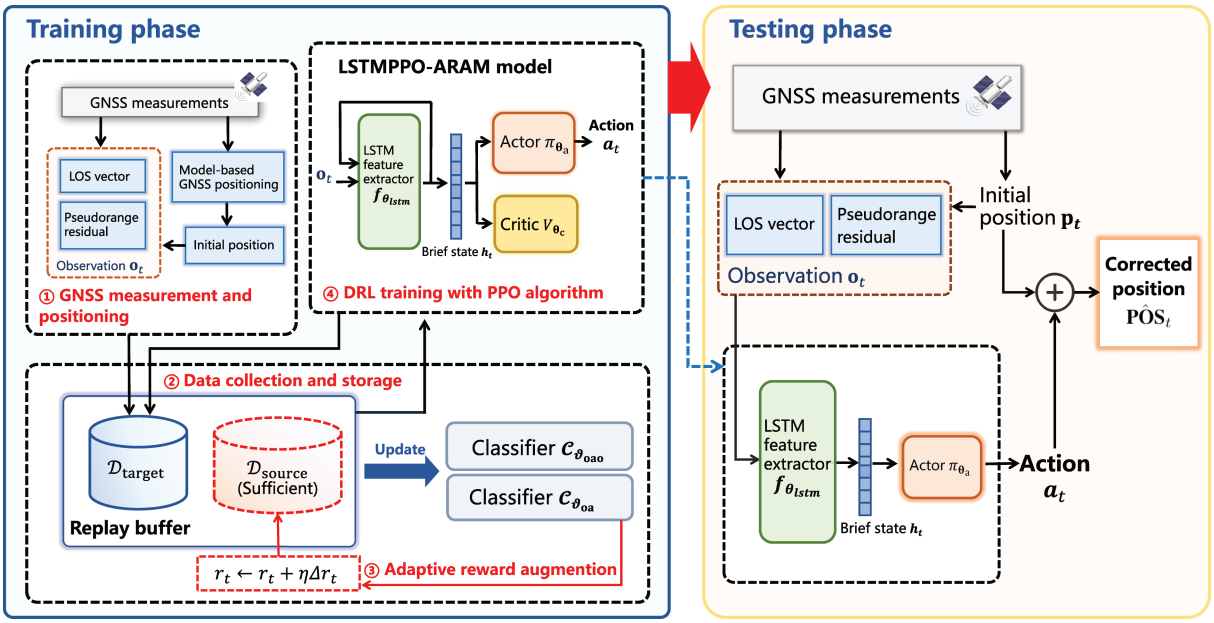


FIGURE 1 Framework of the training phase and testing phase of the proposed LSTMPPO-ARAM method

the initial position without iteration. It is noted that the modules used for ARAM ($\mathcal{C}_{\text{oao}}$ and $\mathcal{C}_{\text{oa}}$) and the critic network are used only in the training phase. The general framework of the LSTMPPO-ARAM method for positioning correction is presented in Figure 1.

4 | EXPERIMENTS

In this section, the proposed LSTMPPO-ARAM algorithm for positioning correction is evaluated on the public GSDC 2022 data set (Fu et al., 2020), which contains real-world data collected in North America from raw Android GNSS measurements. Moreover, our algorithm is also evaluated on the Guangzhou GNSS measurement data set collected by multiple N307-5D receivers in different locations in Guangzhou, China. We present the performance of learned positioning-correction policies in different urban environments. In particular, our goal is to answer the following questions: 1) Can the use of sufficient data collected in the source domain environments with ARAM for domain adaptation effectively improve the performance of DRL? 2) Can the proposed LSTMPPO-ARAM perform better than model-based and DNN-based approaches? 3) How does the performance of the proposed LSTMPPO-ARAM, which introduces GNSS measurement observations, compare with a DRL-based method that uses vehicle position observations?

4.1 | Data Set for Experiments

In this section, we detail the experimental data sets. The GNSS measurement data were collected in various typical scenarios, such as roads on highways, regions surrounded by buildings, and areas under overpasses.

(1) GSDC 2022 data set: The GSDC 2022 data set contains GNSS measurements of vehicle trajectories collected by different Android smartphones with

1-Hz sampling frequency, where trajectories include areas near the San Francisco International Airport and Los Angeles International Airport and each trajectory contains approximately 2,000 positioning time steps. In addition, the data set contains high-precision ground truth positions obtained through the NovAtel SPAN system as reference positions $\mathbf{POS}_t^{\text{ref}}$. We eliminate the trajectories for which the number of visible satellites is zero for more than 50% of the positioning values and divide the remaining trajectories into two scenarios. We then construct the corresponding environments: a semi-urban environment with 44 trajectories, where most of the roads are highways and there are few nearby buildings, and an urban environment with 35 trajectories, where the roads are in dense urban areas and there are many buildings or overpasses. The initial rough positions $\mathbf{POS}_t^{\text{init}}$ are obtained by the baseline KF method, and the GNSS measurements for the observations are formed by L1-frequeny GPS signals. For this data set, we use different methods to correct the initial rough positions in the ECEF coordinate system; that is, we choose $\mathbf{a}_t := \mathbf{a}_t^{\text{ECEF}}$, $r_t := r_t^{\text{ECEF}}$, and $u^{\text{ECEF}} = 2$ for the DRL-based methods.

We further construct the target and source domain environments for positioning correction with semi-urban and urban trajectories. The data for the target domain environments (semi-urban/urban) consist of trajectories collected by different smartphones at a specific location. The data for the source domain environments contain trajectories collected at other different locations, and the remaining semi-urban or urban trajectories are used for evaluation. The trajectory examples of semi-urban and urban target domain environments are shown in Figure 2. Consequently, we construct two target domain environments and corresponding

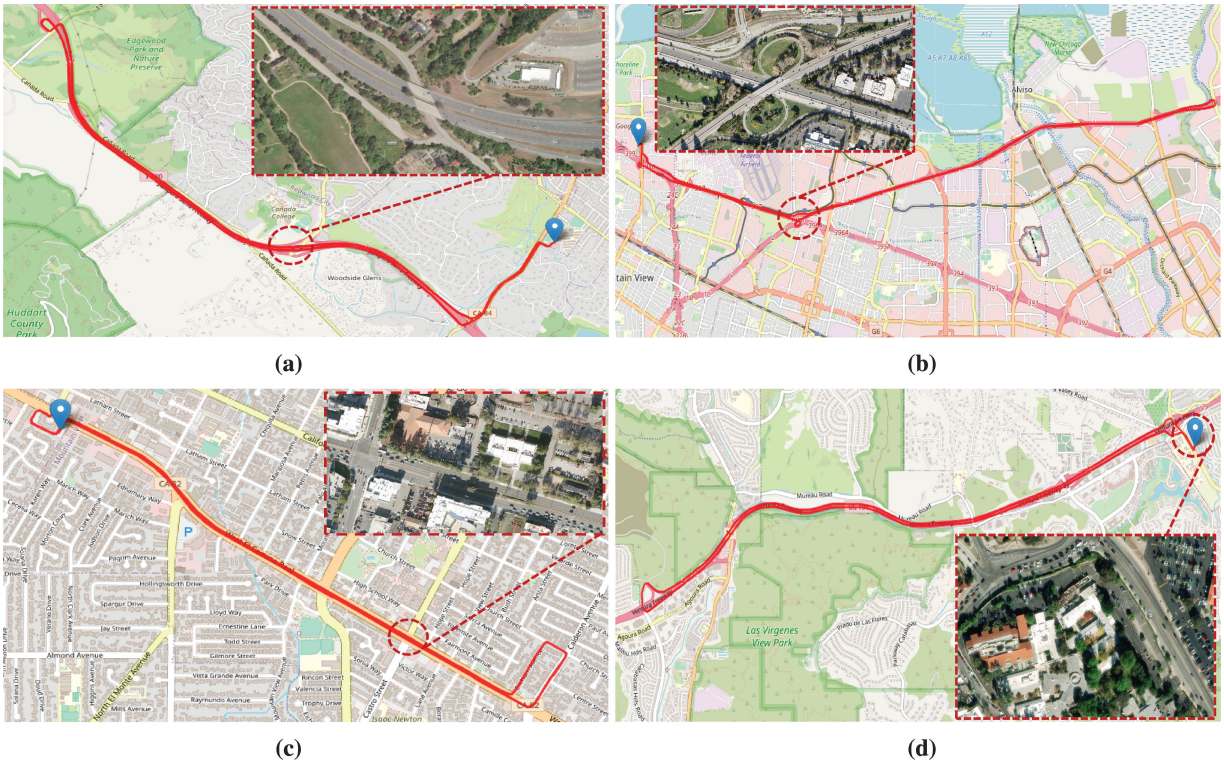


FIGURE 2 Example trajectories in semi-urban and urban environments of the target domain from the GSDC 2022 data set, with corresponding real-world images of each trajectory shown in each inset (a) Highway in San Francisco (SFO, semi-urban) (b) Highway in Mountain View (MTV, semi-urban) (c) Downtown in Mountain View (MTV, urban) (d) Road in Los Angeles (LAX, urban)

TABLE 1
Experimental Details of Environments from the GSDC 2022 Data Set

Target environment	Trajectory number for target	Trajectory number for source	Description
San Francisco (SFO, semi-urban)	7	30	Highways and open roads in suburbs
Mountain View (MTV, semi-urban)	7	31	Highways and open roads in urban fringes
Mountain View (MTV, urban)	3	25	Downtown regions surrounded by buildings
Los Angeles (LAX, urban)	3	17	Roads and overpasses surrounded by towns and forests

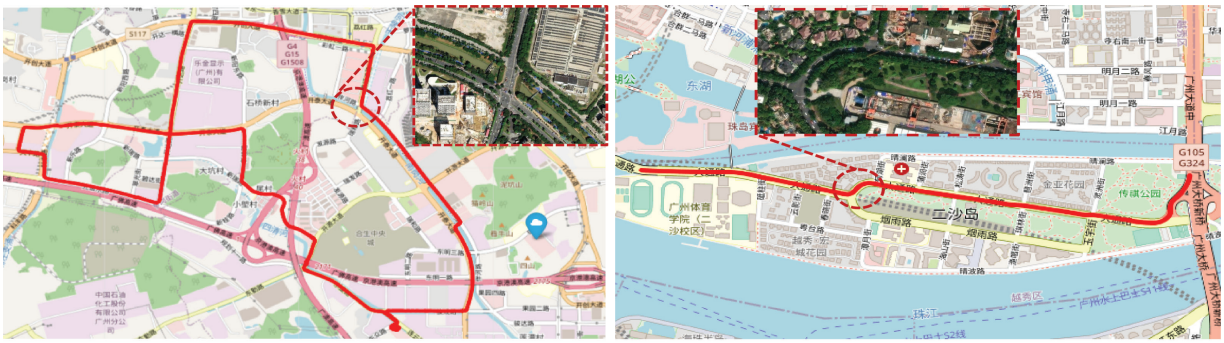
source domain environments for semi-urban and urban scenarios. The details of each environment are presented in Table 1.

(2) Guangzhou GNSS measurement data set: The Guangzhou GNSS measurement data set consists of trajectories collected by multiple N307-5D receivers on roads in different locations in Guangzhou, China. The GNSS measurements contain multiple constellation systems, including GPS, BeiDou, Galileo, and the Quasi-Zenith Satellite System. The sampling frequency is 10 Hz. We used the NovAtel SPAN+LCI system to obtain high-accuracy position estimations as ground truth positions and obtained initial positions using RTK. Because of the large elevation error of the collected data, we only correct the longitude and latitude of the initial positions $\mathbf{POSLL}_t^{\text{init}}$, that is, $\mathbf{a}_t := \mathbf{a}_t^{\text{LL}}$ and $r_t := r_t^{\text{LL}}$, with the scaling factor set to $u^{\text{LL}} = 2e^{-5}$. The data for the target domain environment were collected in the semi-urban Science City with 2603 estimated positions, and the data for the source domain environment were collected along a highway with 20,688 estimated positions. We used the least-square ambiguity decorrelation adjustment (LAMBDA) algorithm (De Jonge & Tiberius, 1996) for RTK ambiguity resolution and the ratio of the second-best root mean square (RMS) to the best RMS to measure the success rate of ambiguity resolution. The default value for the RTK ratio test was set to 3. For the experimental environments, the average ratio of ambiguity resolution in the target domain environment is approximately 25.403, and the average ratio in the other environments is approximately 19.748. After model training, we evaluated the performance of the proposed method on the trajectories in two areas (Semi-urban-1 in Tianhe City and Semi-urban-2 in Ersha Island). Example trajectories from the Guangzhou GNSS measurement data set for training and evaluation are shown in Figure 3, where the blue marker in the plot of Science City is the base station for RTK positioning.

4.2 | Approaches for Comparison

In this section, we detail different model-based algorithms, DNN-based algorithms, and DRL-based algorithms for comparison to evaluate the proposed LSTMPPO-ARAM method as follows:

1. Model-based algorithms: These algorithms include the snapshot positioning WLS method, temporal KF method, and RTK method. The maximum performances for these methods are tuned with Bayesian hyperparameter optimization. We use the KF method as a baseline to obtain the initial positions



(a) (b)

FIGURE 3 Example trajectories for training and evaluation from the Guangzhou GNSS measurement data set, with a corresponding real-world image of each trajectory shown in each inset (a) Example trajectory for training in Science City (b) Example trajectory for evaluation in Semi-urban-2 (Ersha Island)

on the GSDC data set. The RTK method uses GNSS measurements from the base station to correct the GNSS positioning of the receivers. We use the RTK method as the baseline for the Guangzhou GNSS measurement data set.

2. DNN-based algorithms: These algorithms include the deep set (Zaheer et al., 2017), set-transformer (Kanhere et al., 2021), and GCNN (Mohanty & Gao, 2022) methods. The deep set method provides a special deep network structure that can operate on sets, whereas the set-transformer method adopts the transformer architecture for positioning correction with GNSS measurements. The GCNN method predicts the position correction with a graph structure formed using GNSS measurements and KF solutions. The network architectures of the three methods are consistent with the corresponding reference papers, and the network parameters are updated by the Adam method with the trajectories of the above-mentioned target domain environments.
3. DRL-based algorithms: These algorithms include the synchronous advantage actor–critic (A3C) (Zhang & Masoud, 2020), LSTMPPO (Zhao et al., 2023), and proposed LSTMPPO-ARAM methods. The A3C algorithm has a discrete action space and uses the vehicle position as the environment observation. LSTMPPO uses the same setting as LSTMPPO-ARAM. Aside from the fact that ARAM, LSTMPPO, and A3C are only trained in target environments, all algorithms use the same network architecture and initial parameter settings. The hidden layer sizes of the LSTM feature extractor with one layer and the actor–critic network with two layers are $n_{\text{lstm}} = 256$ and $n_{\text{ac}} = 64$, respectively. The rectified linear unit (ReLU) function is used for nonlinear activation, and the network parameters are updated with the Adam optimizer. The maximum absolute value is set to $m = 100$. The replay buffer capacity and discount rate are set to $N = 128$, $\gamma = 0.99$, and the number of training epochs for all collected data in the replay buffer is set to 10. The sampling ratio is set to $r = 5$ for all methods.

4.3 | Parameter Selection and Initialization

In this section, we test the performance of the proposed LSTMPPO-ARAM for varying values of the learning rate lr and temperature parameter η to select

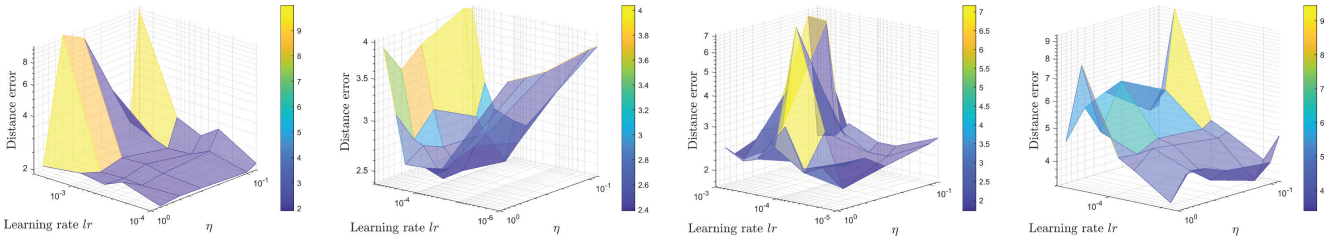


FIGURE 4 Performance of the proposed LSTMPPO-ARAM for a range of η and learning rate lr values in semi-urban and urban environments (a) SFO (semi-urban) (b) MTV (semi-urban) (c) MTV (urban) (d) LAX (urban)

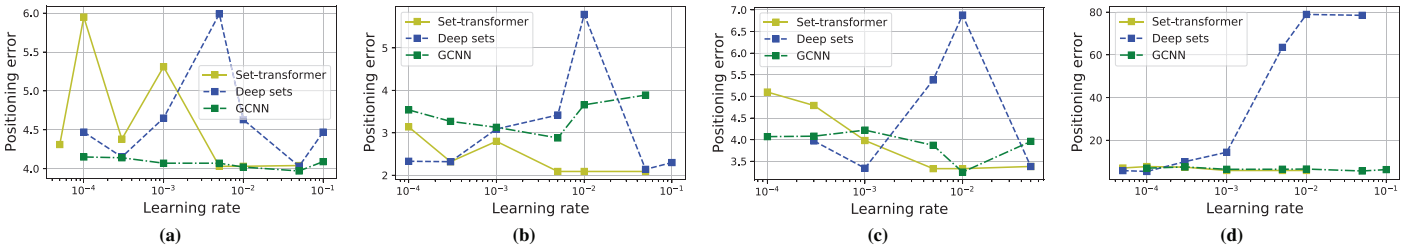


FIGURE 5 Performance of DNN-based methods for a range of learning rate values in semi-urban and urban environments (a) SFO (semi-urban) (b) MTV (semi-urban) (c) MTV (urban) (d) LAX (urban)

optimal parameters for the model in different semi-urban and urban environments on the GSDC 2022 data set. The optimal parameters are searched jointly, as shown in Figure 4, and the evaluation performances of the DNN-based methods (set-transformer, deep sets, and GCNN) over a range of learning rates are shown in Figure 5. The distance error is calculated as $\| \hat{\mathbf{POS}} - \mathbf{POS}^{\text{ref}} \|_2$. The optimal learning rates for LSTMPPO and A3C in different semi-urban and urban environments are searched in the range of $[1e^{-5}, 5e^{-3}]$.

The classifiers $\mathcal{C}_{\text{oa}}$ and $\mathcal{C}_{\text{oao}}$ for ARAM have a fully connected neural network structure with three layers and 256 neurons for the hidden layer. We use the ReLU function for nonlinear activation, the softmax function for output activation, and the Adam method to update network parameters for a learning rate of $\alpha = 3e^{-4}$. To prevent overfitting to a small number of samples, we add Gaussian noise to the inputs of the classifiers. All modeling was performed using Pytorch 1.8 based on a CPU with 256G RAM.

4.4 | Performance Comparison for the GSDC 2022 Data Set

In this section, we compare the proposed LSTMPPO-ARAM against conventional model-based algorithms and state-of-the-art DNN-based and DRL-based algorithms on the GSDC 2022 data set.

4.4.1 | Comparison of Training Performance

With the selected optimal parameters, we compare the training performance of the cumluative rewards and total loss of A3C, LSTMPPO, and the proposed LSTMPPO-ARAM in this section. Figure 6 shows the convergence curves of

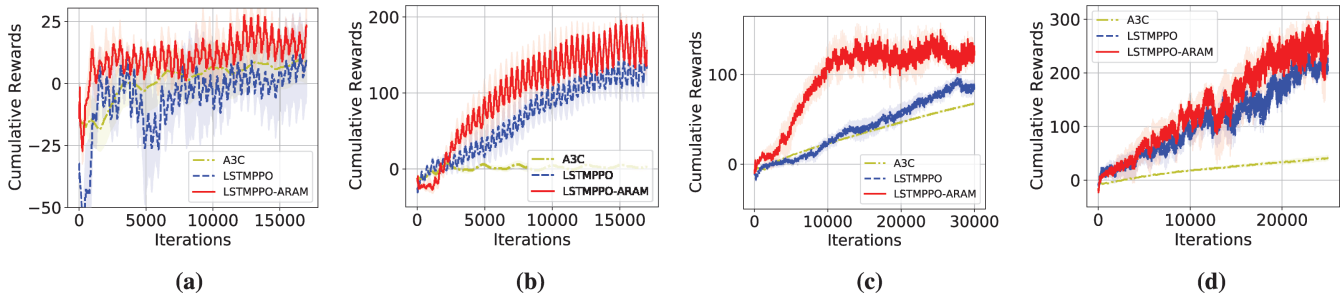


FIGURE 6 Convergence curves of cumulative rewards for training in semi-urban and urban target environments for A3C, LSTMPPO, and LSTMPPO-ARAM with corresponding optimal parameters (a) SFO (semi-urban) (b) MTV (semi-urban) (c) MTV (urban) (d) LAX (urban)

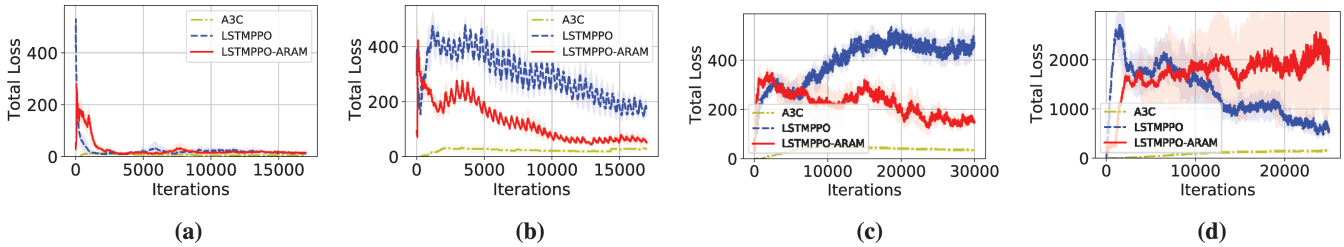


FIGURE 7 Convergence curves of total loss for training in semi-urban and urban target environments for A3C, LSTMPPO, and LSTMPPO-ARAM with corresponding optimal parameters (a) SFO (semi-urban) (b) MTV (semi-urban) (c) MTV (urban) (d) LAX (urban)

cumulative rewards for training in different semi-urban and urban target environments for A3C, LSTMPPO, and LSTMPPO-ARAM with corresponding optimal parameters. Each algorithm was executed five times. The curves have obvious oscillations because of the large differences between different trajectories in the positioning-correction environments. The distributions of positioning errors vary greatly for different trajectories, and the initial rough positions produce large distance errors for some trajectories. In the semi-urban SFO and MTV environments, all methods can converge in 17,000 iterations. Compared with LSTMPPO and A3C, LSTMPPO-ARAM converged in fewer iterations and obtained higher converged cumulative rewards. In the urban MTV environment, LSTMPPO-ARAM required only approximately 10,000 iterations for convergence, whereas LSTMPPO and A3C required more iterations to obtain the converged cumulative rewards. In the urban LAX environment, all methods can converge in 25,000 iterations, although LSTMPPO-ARAM can obtain higher cumulative rewards than LSTMPPO and A3C. Figure 7 shows convergence curves of the total loss for training in semi-urban and urban target environments for LSTMPPO and LSTMPPO-ARAM. In most cases, the proposed LSTMPPO-ARAM can converge to lower loss values than LSTMPPO. Because of the limitation of the discrete action setting of A3C, the loss values rapidly converge and the output positioning-correction accuracy is limited; in contrast, LSTMPPO and LSTMPPO-ARAM, which are based on a continuous action setting, can obtain more accurate correction policies and thus obtain higher cumulative rewards.

Figure 8 shows the reward augmentation (Δr) for training in semi-urban and urban source domain environments. Δr tends to be stable with iterations, indicating that Δr calculated by classifiers can learn an adaptive reward augmentation that compensates for the difference between the reward distributions of the source and target environments, thus achieving domain adaptation from the source domain to the target domain. Overall, the results show that the training

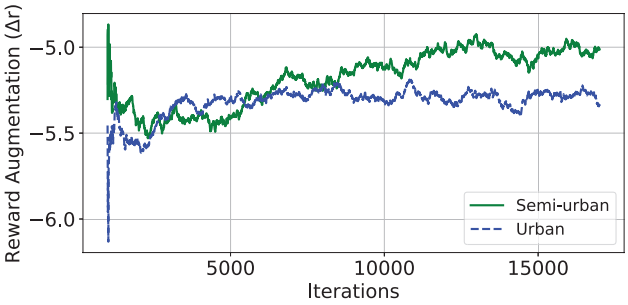


FIGURE 8 Reward augmentation (Δr) for training in semi-urban and urban source domain environments

performance of the DRL model can be effectively improved by using ARAM to transfer experiences from source domain environments to assist an agent learning in a target domain environment.

4.4.2 | *Comparison of Evaluation Performance*

We evaluated the positioning performances of different approaches after training by comparing the positioning errors of trajectories. Tables 2 and 3 present the evaluation performances of different methods with corresponding optimal parameters in semi-urban and urban environments in terms of the average positioning errors along different directions and the total distance in the ECEF coordinate system. The positioning trajectory data were collected in nonstationary driving environments; thus, the variation in positioning error at different positions is large, resulting in a large standard deviation.

Overall, in different semi-urban and urban environments, the proposed LSTMPPO-ARAM obtains the lowest total distance errors, providing a reduction of approximately 15% and 8% in total distance errors from the model-based methods and DNN-based methods, respectively. The LSTMPPO and DNN-based methods can also obtain smaller total distance errors than the model-based methods, but there is still a large positioning error in one direction, which may be due to the inconsistency of GNSS positioning errors in horizontal and vertical directions and the different effects of the correction on the directions. In contrast, LSTMPPO-ARAM has low errors in all directions in the ECEF coordinate system. Overall, the results indicate that ARAM can effectively improve the generalization performance of the DRL model by transferring experiences from the source domain environment with sufficient collected data and thus enhance the positioning accuracy of the model in similar scenes. By introducing GNSS measurements as the observation, LSTMPPO-ARAM can obtain better performance than A3C, which uses only the vehicle position observation; thus, LSTMPPO-ARAM reflects more accurate current states of the agent. Moreover, LSTMPPO-ARAM achieves better performance in all nonstationary environments compared with the DNN-based methods, and the standard deviations of the DNN-based methods are larger than those of the baseline KF and DRL-based methods. This difference may be due to the fact that the DNN-based methods use only independent positioning samples for training without learning the dynamic pattern of the positioning trajectory; thus, these methods may fail in determining the position when the error changes drastically. In contrast, the DRL-based method can effectively learn the dynamic pattern of the positioning error by interacting with the environment, which is more robust to drastic changes in positioning errors, resulting in lower standard deviations.

TABLE 2
Evaluation Performance in Semi-Urban Environments: Average Positioning Errors (Mean±Standard Deviation) Along Different Axes and Total Distance in the ECEF Coordinate System for Different Methods with Corresponding Optimal Parameters

	SFO (Semi-urban)				MTV (Semi-urban)			
	X (m)	Y (m)	Z (m)	Total distance (m)	X (m)	Y (m)	Z (m)	Total distance (m)
WLS	2.28±1.45	2.34±1.68	1.85±1.51	4.17±2.19	1.61±1.48	1.98±2.11	1.76±2.18	3.48±2.99
KF (baseline)	2.21±1.13	2.61±1.18	1.61±1.09	4.21±1.48	0.92±0.79	1.26±1.18	0.90±0.91	2.12±1.10
KF+Deep sets (Zaheer et al., 2017)	2.10±1.53	2.52±1.70	1.57±1.27	4.03±1.97	1.00±0.82	1.27±1.18	0.90±0.89	2.14±1.31
KF+Set-transformer (Kanhere et al., 2021)	2.09±1.57	2.52±1.70	1.57±1.28	4.04±1.96	0.92±0.79	1.26±1.18	0.90±0.91	2.09±1.32
KF+GCNN (Mohanty & Gao, 2022)	**2.07±1.50**	2.50±1.74	**1.56±1.23**	3.99±1.95	1.17±0.91	1.53±1.33	1.39±1.04	2.70±1.44
KF+A3C (Zhang & Masoud, 2020)	2.12±1.62	2.59±1.17	1.62±1.09	4.16±1.47	1.02±0.73	1.18±1.01	0.99±0.80	2.13±1.12
KF+LSTMPPO (Zhao et al., 2023)	2.09±1.14	2.43±1.17	1.57±1.08	3.96±1.45	0.95±0.72	**1.06±0.96**	**0.87±0.78**	1.91±1.10
KF+LSTMPPO-ARAM	2.08±1.19	**2.12±1.24**	1.61±1.13	**3.77±1.53**	**0.86±0.71**	1.07±0.95	0.91±0.79	**1.87±1.03**

TABLE 3
Evaluation Performance in Urban Environments: Average Positioning Errors (Mean±Standard Deviation) Along Different Axes and Total Distance in the ECEF Coordinate System for Different Methods with Corresponding Optimal Parameters

	MTV (Urban)				LAX (Urban)			
	X (m)	Y (m)	Z (m)	Total distance (m)	X (m)	Y (m)	Z (m)	Total distance (m)
WLS	2.28±2.03	3.51±2.56	2.65±2.45	5.61±3.22	2.72±1.91	3.83±2.66	3.76±2.31	6.73±2.77
KF (baseline)	1.25±0.94	2.45±1.07	1.26±0.77	3.40±1.12	**2.29±1.37**	3.25±1.86	3.20±1.49	5.63±1.73
KF+Deep sets (Zaheer et al., 2017)	1.36±0.98	2.06±1.36	1.74±1.42	3.34±1.68	2.50±1.79	2.75±1.95	2.96±1.83	5.40±1.97
KF+Set-transformer (Kanhere et al., 2021)	1.17±0.99	**1.93±1.70**	1.94±1.44	3.33±1.94	3.19±2.36	**2.57±2.05**	2.75±2.46	5.81±2.53
KF+GCNN (Mohanty & Gao, 2022)	**1.16±0.89**	2.19±1.20	**1.23±0.99**	3.09±1.16	2.44±1.85	2.79±1.83	**2.75±2.07**	5.29±2.09
KF+A3C (Zhang & Masoud, 2020)	1.26±0.95	2.03±1.09	1.46±0.73	3.14±0.97	2.44±1.72	2.85±1.69	3.10±1.72	5.39±1.84
KF+LSTMPPO (Zhao et al., 2023)	1.30±1.33	2.07±1.10	1.34±0.73	3.13±1.01	2.45±1.73	2.70±1.65	3.03±1.69	5.26±1.79
KF+LSTMPPO-ARAM	1.25±0.90	2.06±1.14	1.36±0.78	**3.09±1.03**	2.38±1.59	2.67±1.64	2.96±1.78	**5.11±1.73**

To visually demonstrate the performance of the proposed method, Figure 9 shows the positioning distance errors of trajectories collected by different smartphones in different target semi-urban and urban environments. The positioning errors of trajectories collected by different phones vary greatly, and the error distribution for a given trajectory can show large differences. The results show that LSTMPPO-ARAM has a significant improvement over the baseline KF method and can obtain lower positioning errors in semi-urban and urban environments.

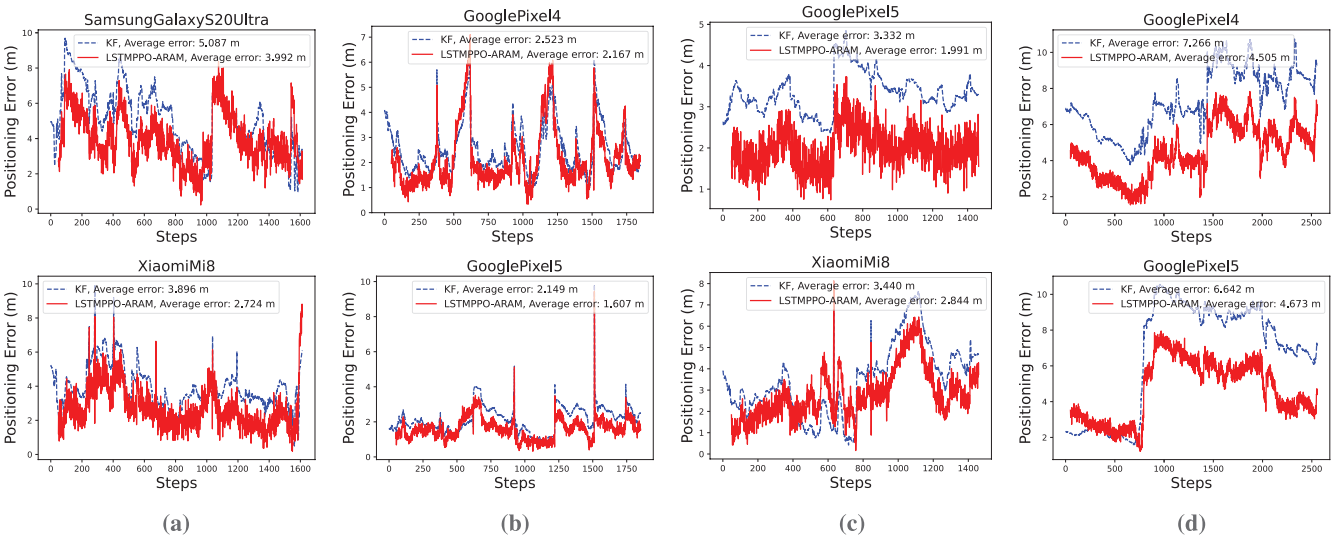


FIGURE 9 Evaluation performance of the positioning distance error of trajectories collected by different types of mobile phones in different target semi-urban and urban environments (a) SFO (semi-urban) (b) MTV (semi-urban) (c) MTV (urban) (d) LAX (urban)

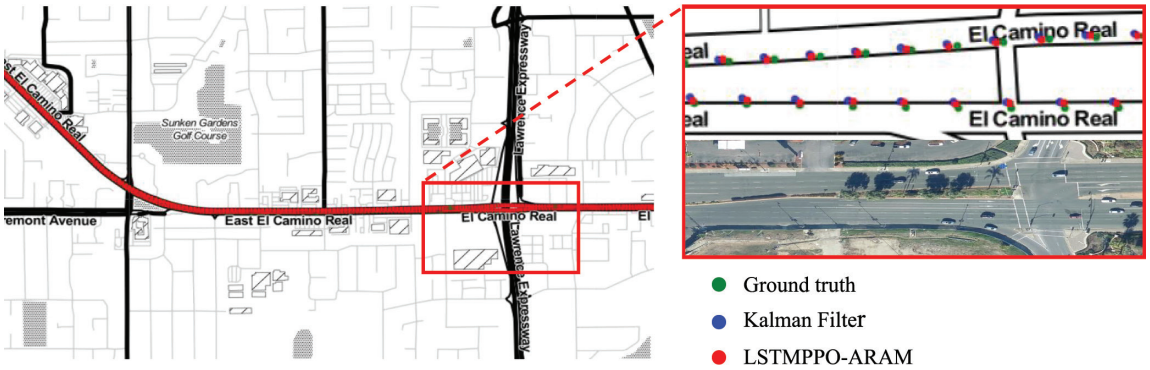


FIGURE 10 Positioning performance for one testing trajectory in MTV (urban)
The left panel shows the overall trajectory, and the right panel provides a detailed view of the positioning trajectory and a corresponding real-world image.

This improvement is particularly notable in urban environments, indicating that LSTMPPO-ARAM can learn effective positioning-correction policies from data collected by different smartphones in different scenarios. In addition, Figure 10 presents a testing positioning trajectory in the MTV (urban) environment for LSTMPPO-ARAM and the KF method; here, it can be intuitively seen that the positioning results obtained by the proposed method are closer to the reference positions of ground truth than those obtained by the KF method, indicating that the proposed LSTMPPO-ARAM can effectively correct the initial rough positioning results and thus improve the positioning accuracy.

4.5 | Validation on the Guangzhou GNSS Measurement Data Set

We further applied our algorithm to our collected Guangzhou GNSS measurement data set for positioning correction. We first present the distribution of the number of visible satellites for different environments in Figure 11. It can be seen

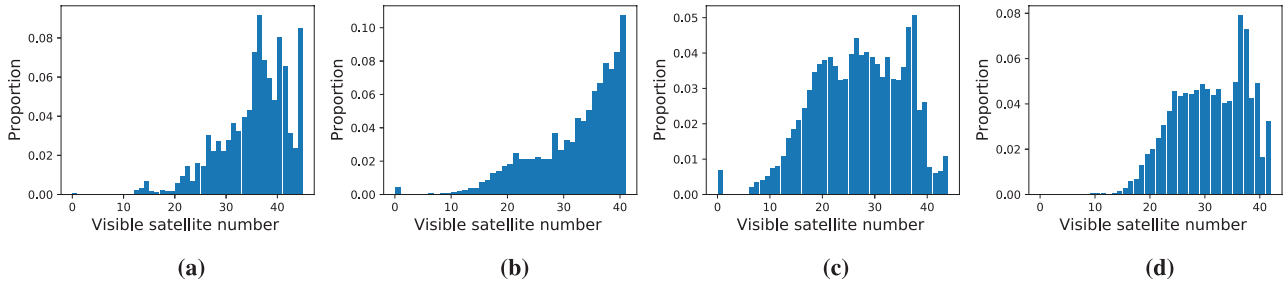


FIGURE 11 Distribution of the number of visible satellites for Science City (target domain), a highway (source domain), and two environments for evaluation (Semi-urban-1 and Semi-urban-2) from the Guangzhou GNSS measurement data set (a) Science City (target domain) (b) Highway (source domain) (c) Semi-urban-1 (evaluation) (d) Semi-urban-2 (evaluation)

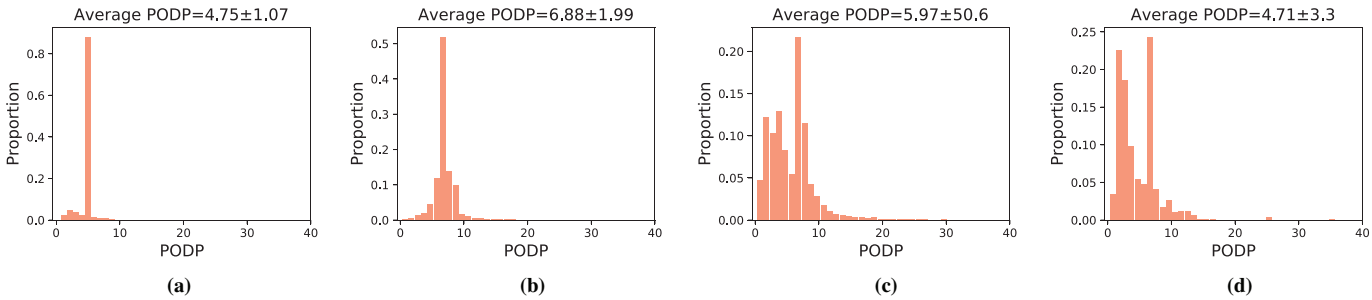


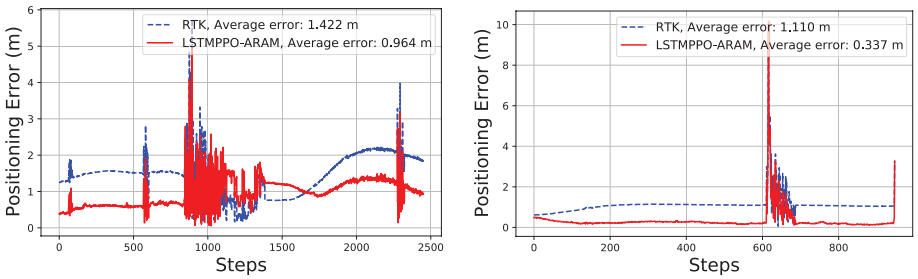
FIGURE 12 Distribution of the PODP for Science City (target domain), a highway (source domain), and two environments for evaluation (Semi-urban-1 and Semi-urban-2) from the Guangzhou GNSS measurement data set (a) Science City (target domain) (b) Highway (source domain) (c) Semi-urban-1 (evaluation) (d) Semi-urban-2 (evaluation)
The mean and standard deviation of the PODP are shown at the top of each plot.

that the receiver can observe more than 10 visible satellites in most cases. For a few cases in which the multi-path effect is severe, only 2–3 satellites can be received, which can lead to a large deviation in the RTK positioning solution. Moreover, Figure 12 presents the PODP of different environments. The calculation of position dilution of precision (PDOP) can be defined as $\text{PODP} = \sqrt{\sigma_x^2 + \sigma_y^2 + \sigma_z^2}$, where σ_x, σ_y, and σ_z are the biases in three directions. It can be seen that the PODP is less than 10 in most cases. There are more outliers in Semi-urban-1 than in the other environments, which can make it difficult for the model training to converge. To avoid instability in training, we eliminate the outliers in the data set. Additionally, dominant elevation errors are mostly at the meter level, while horizontal errors are mostly at the submeter level. Considering that the horizontal position can meet the basic requirements for GNSS positioning, we only consider the distance errors of latitude and longitude in this data set.

To verify the positioning performance of the proposed method, we compare the total distance error in latitude and longitude for the RTK method and the proposed LSTMPPO-ARAM method, shown in Table 4, where the distance error is calculated by $\text{Vincenty}(\hat{\mathbf{POSLL}}, \mathbf{POSLL}^{\text{ref}})$. The results indicate that the proposed LSTMPPO-ARAM can reduce the positioning errors of initial positions obtained by the RTK method in both training and evaluation. In addition, Figure 13 shows qualitative positioning error results for the RTK and LSTMPPO-ARAM methods via sample trajectory plots, which indicate that, even for nonstationary trajectories with dynamic changes in positioning errors, the proposed LSTMPPO-ARAM can quickly follow changes in error and determine an adaptive dynamic correction policy for positioning correction.

TABLE 4
Performance for the Guangzhou GNSS Measurement Data Set: Distance Errors (Mean±Standard Deviation) in Longitude and Latitude for the RTK and LSTMPPO-ARAM Methods with Corresponding Optimal Parameters

	Science City (training)	Semi-urban-1 (evaluation)	Semi-urban-2 (evaluation)
RTK (baseline)	0.425±0.854 (m)	0.782±0.461 (m)	0.813±0.536 (m)
RTK+LSTMPPO-ARAM	**0.398±1.067** (m)	**0.688±0.384** (m)	**0.726±0.435** (m)



(a) **(b)**

FIGURE 13 Trajectory plots of the positioning distance error in longitude and latitude for the RTK method and the proposed LSTMPPO-ARAM method in the Semi-urban-1 and Semi-urban2 environments for evaluation (a) Semi-urban-1 (b) Semi-urban-2

4.6 | Computational Complexity Analysis

In this section, we analyze the computational complexity of the proposed method. We first denote the hidden layer sizes of the LSTM feature extractor and actor network as n_{lstm} and n_{ac}; the hidden layer sizes of the set-transformer and GCNN are denoted as n_{set} and n_{gcnn}, and the dimension of the input observation is denoted as $d = 4S_{max}$. For DRL-based positioning methods, the computational complexity of LSTMPPO and LSTMPPO-ARAM, which share the same model structure, is $O(n_{\text{lstm}}^2)$. The computational complexity of A3C is $O(n_{ac}n_{action})$, where n_{action} is the dimension of the discrete action space. For DNN-based positioning methods, the computations for the GCNN and set-transformer methods are $O(n_{\text{set}}d^2)$ and $O(n_{\text{gcnn}}^2 d)$. In the training phase, the time cost for all training epochs using all data in the replay buffer is approximately 0.2 s, indicating that, after sampling all of the data each time, it takes only a short time to complete the training of all stored data. In the testing phase without data storage, the time costs of each output positioning correction for the DRL-based and DNN-based methods are approximately 0.001 s and 0.002 s, which are much lower than the frequency of typical GNSS positioning solutions (e.g., 1 Hz or 50 Hz) and can thus be ignored. Therefore, the integration of the proposed LSTMPPO-ARAM and GNSS can still maintain real-time operation in real-world applications.

4.7 | Discussion

In the experiment section, the proposed algorithm LSTMPPO-ARAM for positioning correction was evaluated on the GSDC 2022 and Guangzhou GNSS measurement data sets and compared with conventional model-based algorithms and

state-of-the-art DNN-based and DRL-based algorithms, aiming to answer the questions posed at the beginning of Section 4.

1. We compared the training performance of LSTMPPO-ARAM and LSTMPPO. As shown in Figures 6 and 7, LSTMPPO-ARAM can obtain higher cumulative rewards with lower loss values and can converge in fewer iterations. Tables 2 and 3 highlight the superior testing performance of LSTMPPO-ARAM versus LSTMPPO, demonstrating that the performance of DRL can be effectively improved by employing ARAM to transfer experiences from source domain environments for training.
2. We compared LSTMPPO-ARAM with model-based and DNN-based methods. As shown in Tables 2 and 3, LSTMPPO-ARAM can achieve better performance in all urban environments. Table 4 and Figure 13 show that the positioning-correction policy of LSTMPPO-ARAM is effective even in nonstationary urban environments with drastic dynamic changes, illustrating the adaptability of LSTMPPO-ARAM to nonstationary urban environments.
3. We presented the training performance of different DRL-based methods in Figure 6 and evaluated the positioning performances of different approaches after training in Tables 2 and 3. The results show that LSTMPPO-ARAM can outperform the A3C method, which uses the vehicle position observation, in terms of cumulative rewards and positioning errors in evaluation, demonstrating the effectiveness of the GNSS measurement observations used in LSTMPPO-ARAM.

5 | CONCLUSION

In this work, a novel DRL-based positioning-correction approach, LSTMPPO-ARAM, was proposed to improve the GNSS positioning accuracy in nonstationary urban environments. To address the poor performance of DRL caused by inadequate data for training in the target domain environment, we employed ARAM to adaptively modify data matching between the source domain and target domain for nonstationary urban environments, to transfer training experiences from the source domain environment with sufficient data, and to compensate for the performance degradation caused by limited training data. Overall, we constructed a DRL model, LSTMPPO-ARAM, based on ARAM that uses GNSS measurement observations to achieve an adaptive dynamic positioning-correction policy. Moreover, the proposed positioning algorithm can be flexibly combined with different positioning methods, such as single point positioning KF and differential positioning RTK methods.

The proposed method was evaluated on different positioning-correction environments constructed using the GSDC 2022 and Guangzhou GNSS measurement data sets. We compared the proposed LSTMPPO-ARAM with conventional model-based algorithms as well as state-of-the-art DNN-based and DRL-based algorithms. The results show that the proposed method can learn effective positioning-correction policies using data collected in different locations by different receivers, correct the initial positions along three axes in the ECEF coordinate system or the longitude and latitude of the positions, and thus improve the positioning accuracy in nonstationary urban environments. The proposed LSTMPPO-ARAM can obtain a performance improvement of approximately 10% over model-based methods and approximately 8% over DNN-based and DRL-based approaches. In future work, we will attempt to enhance the generalization capability of the proposed model across

different environments and extend the validation to a broader range of data sets, such as data sets collected in urban canyon environments.

ACKNOWLEDGMENTS

This research was supported in part by the National Natural Science Foundation of China under grants 62273106, 62203122, 62320106008, 62373114, and 62203123, in part by the GuangDong Basic and Applied Basic Research Foundation under grants 2023A1515011480 and 2023A1515011159, and in part by the National Key Research and Development Plan-Strategic Scientific and Technological Innovation Cooperation Key Project under grant 2023YFE0209400.

REFERENCES

Bo, Z., Qiang, L., Guoliang, X., Renlan, C., & Feng, C. (2022). Application of GNSS+INS integrated navigation in mine auto-driving trucks. *Bulletin of Surveying and Mapping*, (7), 143. https://doi.org/10.13474/j.cnki.11-2246.2022.0219

De Jonge, P., & Tiberius, C. (1996). The LAMBDA method for integer ambiguity estimation: Implementation aspects. *Publications of the Delft Computing Centre, LGR-Series*, *12*(12), 1–47.

Eysenbach, B., Asawa, S., Chaudhari, S., Levine, S., & Salakhutdinov, R. (2020). Off-dynamics reinforcement learning: Training for transfer with domain classifiers. *arXiv*. https://doi.org/10.48550/arXiv.2006.13916

Fu, G. M., Khider, M., & van Diggelen, F. (2020). Android raw GNSS measurement datasets for precise positioning. *Proc. of the 33rd International Technical Meeting of the Satellite Division of the Institute of Navigation (ION GNSS+ 2020)*, 1925–1937. https://doi.org/10.33012/2020.17628

Han, K., Lee, S., Song, Y.-J., Lee, H.-B., Park, D.-H., & Won, J.-H. (2021). Precise positioning with machine learning based Kalman filter using GNSS/IMU measurements from Android smartphone. *Proc. of the 34th International Technical Meeting of the Satellite Division of the Institute of Navigation (ION GNSS+ 2021)*, St. Louis, MO, 3094–3102. https://doi.org/10.33012/2021.18005

Kanhere, A. V., Gupta, S., Shetty, A., & Gao, G. (2021). Improving GNSS positioning using neural network-based corrections. *Proc. of the 34th International Technical Meeting of the Satellite Division of the Institute of Navigation (ION GNSS+ 2021)*, St. Louis, MO, 3068–3080. https://doi.org/10.33012/2021.17999

Kingma, D., & Ba, J. (2015). Adam: A method for stochastic optimization. *Proc. of the 3rd International Conference on Learning Representations,* San Diego, CA. https://doi.org/10.48550/arXiv.1412.6980

Le, L., Kumaraswamy, R., & White, M. (2017). Learning sparse representations in reinforcement learning with sparse coding. *arXiv*. https://doi.org/10.48550/arXiv.1707.08316

Li, H., Borhani-Darian, P., Wu, P., & Closas, P. (2020). Deep learning of GNSS signal correlation. *Proc. of the 33rd International Technical Meeting of the Satellite Division of the Institute of Navigation (ION GNSS+ 2020)*, 2836–2847. https://doi.org/10.33012/2020.17598

Li, S., Mikhaylov, N., & Pany, T. (2023). Performance analysis of deep learning supported Kalman filter. *Proc. of the 2023 International Technical Meeting of the Institute of Navigation*, Long Beach, CA, 1101–1109. https://doi.org/10.33012/2023.18640

Li, Z., Zeng, K., Wang, L., Kan, X., Yuan, R., & Xie, S. (2023). NLOS/LOS classification by constructing indirect environment interaction from GNSS receiver measurements using a transformer-based deep learning model. *Proc. of the 2023 International Technical Meeting of the Institute of Navigation*, Long Beach, CA, 859–870. https://doi.org/10.33012/2023.18657

Liu, J., & Guo, G. (2021). Vehicle localization during GPS outages with extended Kalman filter and deep learning. *IEEE Transactions on Instrumentation and Measurement*, *70*, 1–10. https://doi.org/10.1109/TIM.2021.3097401

Liu, J., Zhang, H., & Wang, D. (2022). DARA: Dynamics-aware reward augmentation in offline reinforcement learning. *arXiv*. https://doi.org/10.48550/arXiv.2203.06662

Mihatsch, O., & Neuneier, R. (2002). Risk-sensitive reinforcement learning. *Machine Learning*, *49*, 267–290. https://doi.org/10.1023/A:1017940631555

Min, D., Kim, M., Lee, J., Circiu, M. S., Meurer, M., & Lee, J. (2022). DNN-based approach to mitigate multipath errors of differential GNSS reference stations. *IEEE Transactions on Intelligent Transportation Systems*, *23*(12), 25047–25053. https://doi.org/10.1109/TITS.2022.3207281

Mohanty, A., & Gao, G. (2022). Learning GNSS positioning corrections for smartphones using graph convolution neural networks. *Proc. of the 35th International Technical Meeting of the Satellite Division of the Institute of Navigation (ION GNSS+ 2022)*, Denver, CO, 2215–2225. https://doi.org/10.33012/2022.18372

Mohanty, A., & Gao, G. (2023). Tightly coupled graph neural network and Kalman filter for smartphone positioning. *Proc. of the 36th International Technical Meeting of the Satellite Division of the Institute of Navigation (ION GNSS+ 2023)*, Denver, CO, 175–187. https://doi.org/10.33012/2023.19300

Niu, X., Tang, H., Zhang, T., Fan, J., & Liu, J. (2022). IC-GVINS: A robust, real-time, INS-centric GNSS-visual-inertial navigation system. *IEEE Robotics and Automation Letters*, *8*(1), 216–223. https://doi.org/10.1109/LRA.2022.3224367

Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2015). High-dimensional continuous control using generalized advantage estimation. *arXiv*. https://doi.org/10.48550/arXiv.1506.02438

Sun, R., Fu, L., Cheng, Q., Chiang, K.-W., & Chen, W. (2023). Resilient pseudorange error prediction and correction for GNSS positioning in urban areas. *IEEE Internet of Things Journal*, *10*(11). https://doi.org/10.1109/JIOT.2023.3235483

Vincenty, T. (1975). Direct and inverse solutions of geodesics on the ellipsoid with application of nested equations. *Survey Review, 23*(176), 88–93. https://doi.org/10.1179/sre.1975.23.176.88

Wang, S., Bao, Z., Culpepper, J. S., & Cong, G. (2021). A survey on trajectory data management, analytics, and learning. *ACM Computing Surveys (CSUR)*, *54*(2), 1–36. https://doi.org/10.1145/3440207

Yuan, H., Zhang, Z., He, X., Wen, Y., & Zeng, J. (2022). An extended robust estimation method considering the multipath effects in GNSS real-time kinematic positioning. *IEEE Transactions on Instrumentation and Measurement*, *71*, 1–9. https://doi.org/10.1109/TIM.2022.3193967

Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., & Smola, A. J. (2017). Deep sets. *Advances in Neural Information Processing Systems*, *30*. https://proceedings.neurips.cc/paper/2017/hash/f22e4747da1aa27e363d86d40ff442fe-Abstract.html.

Zhang, E., & Masoud, N. (2020). Increasing GPS localization accuracy with reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems*, *22*(5), 2615–2626. https://doi.org/10.1109/TITS.2020.2972409

Zhang, Z., Zhang, D., & Qiu, R. C. (2019). Deep reinforcement learning for power system applications: An overview. *CSEE Journal of Power and Energy Systems*, *6*(1), 213–225. https://doi.org/10.17775/CSEEJPES.2019.00920

Zhao, H., Li, Z., Chen, C., Wang, L., Xie, K., & Xie, S. (2023). Fusing vehicle trajectories and GNSS measurements to improve GNSS positioning correction based on actor-critic learning. *Proc. of the 2023 International Technical Meeting of the Institute of Navigation*, Long Beach, CA, 82–94. https://doi.org/10.33012/2023.18593

Zhao, H., Wang, J., Huang, X., Li, Z., & Xie, S. (2022). Dictionary learning-based reinforcement learning with non-convex sparsity regularizer. *Proc. of the Artificial Intelligence: Second CAAI International Conference (CICAI 2022),* Beijing, China, 81–93. https://doi.org/10.1007/978-3-031-20503-3_7

Zhao, H., Wu, J., Li, Z., Chen, W., & Zheng, Z. (2022). Double sparse deep reinforcement learning via multilayer sparse coding and nonconvex regularized pruning. *IEEE Transactions on Cybernetics, 53*(2), 765–778. https://doi.org/10.1109/TCYB.2022.3157892

Zhu, H., Xia, L., Li, Q., Xia, J., & Cai, Y. (2022). IMU-aided precise point positioning performance assessment with smartphones in GNSS-degraded urban environments. *Remote Sensing*, *14*(18), 4469. https://doi.org/10.3390/rs14184469

How to cite this article: Tang, J., Li, Z., Hou, K., Li, P., Zhao, H., Wang, Q., Liu, M., & Xie, S. (2024). Improving GNSS positioning correction using deep reinforcement learning with an adaptive reward augmentation method. *NAVIGATION, 71*(4). https://doi.org/10.33012/navi.667